Software Requirements Specification (SRS)

Remote Lab Reservation System

Project	Remote Lab Reservation System (RLRS)
Version	1.0
Authors	Four-member Software Engineering project team
Date	30 October 2026
Status	Draft for team review

Revision History

Version	Date	Author	Change summary	Approval
1.0	30 October 2026	Four-member Software Engineering project team	Initial project-specific draft	Pending

Approvals

Role	Name	Signature / Email	Date
Project team representative	To be filled by team	Pending	
Course instructor / TA	To be filled by instructor	Pending	

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features / Functional Requirements
- 5. Non-functional Requirements
- 6. Security Objectives and Requirements
- 7. Quality Attributes and Acceptance Tests
- 8. UML Use-Case Models
- 9. Requirements Traceability Matrix

1. Introduction

1.1 Purpose

This SRS defines the functional and non-functional requirements for the Remote Lab Reservation System (RLRS), a booking portal through which students reserve timeslots on lab equipment, view calendar availability, and receive automated email reminders. It is the reference for implementation, testing, and project evaluation.

1.2 Scope

RLRS supports managing labs and equipment, configuring bookable hours and slot lengths, showing availability on a calendar, creating, approving and cancelling reservations, detecting and preventing scheduling conflicts, blocking slots for maintenance, sending confirmation and reminder emails, recording attendance, and reporting on equipment utilisation. The initial project is a software prototype for one institution. It does not control lab equipment directly, integrate with university timetable or student information systems, take payments, or send SMS/push notifications unless added and approved later.

1.3 Audience

Student developers, testers, users acting as students, faculty and lab in-charges, project evaluators, and the teaching assistant.

1.4 Definitions

Term	Definition
RLRS	Remote Lab Reservation System
Equipment	A single bookable lab resource (instrument, workstation, kit or remote-access setup) belonging to one lab
Slot	A bookable time interval on the lab's calendar grid, defined by the lab's operating hours and slot length
Reservation	A booking of one equipment item by one user for a start and end time, with a status
Conflict	Two active reservations, or a reservation and a maintenance block, on the same equipment whose time ranges overlap
Maintenance block	A time range during which equipment is unavailable for booking
Booking rules	Configurable limits such as advance-booking window, maximum duration, maximum active bookings per student, and cancellation cut-off
No-show	A confirmed reservation the user did not attend
REST	Representational State Transfer; resource-oriented HTTP API style
FR / NFR	Functional Requirement / Non-functional Requirement
RBAC	Role-Based Access Control

2. Overall Description

2.1 Product Perspective

RLRS is a web application with a calendar-based user interface, a RESTful API, a scheduling engine, a background job scheduler for emails, and a persistent database. Users browse equipment availability and submit booking requests; the scheduling engine validates them against existing reservations, maintenance blocks and booking rules; confirmed bookings are stored and trigger confirmation and reminder emails.

2.2 Major Product Functions

- Authenticate users and apply role-based permissions.
- Create and maintain labs, equipment, operating hours and slot lengths.
- Show equipment availability in day and week calendar views.
- Create reservations on free slots, with conflict detection and booking-rule checks.
- Route reservations for restricted equipment to a lab in-charge for approval.
- Let users view and cancel their reservations within the cancellation cut-off.
- Block equipment for maintenance and notify affected users.
- Send confirmation, cancellation and automated reminder emails.
- Record attendance and no-shows after each slot.
- Generate utilisation, booking and no-show reports.

2.3 User Roles and Characteristics

Role	Needs / permissions
Administrator	Manage users and roles; manage labs; configure booking rules and email settings; view all reservations, reports and audit history.
Lab in-charge	Manage equipment in their lab; set operating hours and slot length; approve or reject restricted bookings; create maintenance blocks; mark attendance; view lab reports.
Faculty	Book equipment for themselves or a class session within faculty limits; view lab calendars.
Student	View availability; create, view and cancel their own reservations; receive emails.
Viewer / evaluator	Read calendars and reports without making or changing bookings.

2.4 Operating Environment

The prototype is intended to run in a modern desktop or laptop browser, with a Python-based REST API and scheduling layer, a relational database, a background job scheduler, and an SMTP email service (or a local test mail server during development). The application should support a local development environment and a reproducible setup documented in the repository.

2.5 Constraints and Assumptions

- Each reservation is for one equipment item; booking several items means several reservations.
- Reservations start and end on the lab's slot boundaries.
- Back-to-back reservations (one ending exactly when the next starts) are not a conflict.
- Users have a valid institutional email address; RLRS does not guarantee delivery once a message is handed to the email service.
- All times are stored in UTC and shown and checked in the institution's configured time zone.
- The system serves one institution in the initial release unless multi-campus support is explicitly implemented.
- The implementation and selected technologies must match the actual repository; record deviations in the final demo/documentation.

3. External Interface Requirements

3.1 User Interface

- Calendar widget with day and week views per lab and per equipment, showing free, booked, own-booking and maintenance slots.
- Selecting a free slot opens a booking form pre-filled with equipment and time.
- "My reservations" page listing upcoming and past bookings with cancel actions where allowed.
- Lab in-charge pages for equipment, operating hours, maintenance blocks, pending approvals and attendance.
- Forms with required-field validation and clear error messages, including the reason a slot cannot be booked.
- Slot states that do not rely on colour alone (text label or icon as well).

3.2 Software Interfaces

- A RESTful JSON API exposes labs, equipment, availability, reservations, maintenance blocks, settings and reports.
- The database stores users, roles, labs, equipment, reservations, maintenance blocks, email jobs, settings and audit events.
- An SMTP service (or transactional email provider) sends confirmation, cancellation and reminder emails.
- Calendar export (iCal), SMS, push notifications and university SSO are outside the initial scope unless implemented and documented.

3.3 Communications

For local development, browser-to-server communication may use localhost and a local test mail server. Any deployed environment should use HTTPS, protect credentials, session tokens and SMTP secrets, and avoid exposing the database directly to end users.

4. System Features / Functional Requirements

Req ID	Requirement (shall)	Priority	Acceptance criteria
RLRS-F-001	The system shall allow an authorized user to sign in and sign out.	High	Valid credentials create a session; invalid credentials are rejected.
RLRS-F-002	The system shall enforce role-based permissions for student, faculty, lab in-charge and administrative actions.	High	A user without permission cannot access or execute a restricted action.
RLRS-F-003	The system shall allow authorized users to create, edit and deactivate labs and equipment, including name, lab, type, description, status and whether approval is required.	High	Valid equipment is saved and appears in browse/search; deactivated equipment cannot be booked.
RLRS-F-004	The system shall allow a lab in-charge to configure each lab's operating hours and slot length.	High	The calendar shows bookable slots only inside operating hours, at the configured length.
RLRS-F-005	The system shall allow users to browse and search equipment by lab, type and availability on a chosen date.	Medium	Results match the selected criteria.
RLRS-F-006	The system shall display equipment availability in day and week calendar views.	High	Each slot is shown as free, booked, own booking or maintenance, matching stored data.
RLRS-F-007	The system shall allow a signed-in user to create a reservation for one equipment item with a start and end time on slot boundaries.	High	A valid request is saved with status Confirmed (or Pending for restricted equipment).
RLRS-F-008	The scheduling engine shall reject a reservation that overlaps an active reservation	High	Overlapping requests are rejected with the conflicting time shown; back-to-back bookings

	or maintenance block on the same equipment.		are accepted.
RLRS-F-009	The scheduling engine shall enforce booking rules: future time, inside operating hours, within the advance-booking window, not above maximum duration, not above the user's maximum active bookings, and no overlapping bookings by the same user.	High	Each rule violation is rejected with a message naming the rule.
RLRS-F-010	The system shall place reservations for approval-required equipment in Pending status and allow the lab in-charge to approve or reject them, with a reason required for rejection.	Medium	Approval confirms the booking; rejection frees the slot and records the reason.
RLRS-F-011	The system shall show each user their upcoming and past reservations with status.	High	The list matches the user's stored reservations.
RLRS-F-012	The system shall allow a user to cancel their own reservation before the configured cancellation cut-off.	High	Cancellation frees the slot immediately; cancellation after the cut-off is refused.
RLRS-F-013	The system shall allow a lab in-charge to create maintenance blocks and shall cancel and notify the holders of any reservations inside the block.	Medium	Blocked slots show as maintenance; affected users receive a cancellation email.
RLRS-F-014	The system shall send an email when a reservation is confirmed, approved, rejected or cancelled.	High	One email per event is queued to the reservation holder with booking details.
RLRS-F-015	The system shall send an automated reminder email a configurable time before each confirmed reservation starts (default 2 hours).	High	The reminder is sent once, within the delivery window, only for reservations still confirmed.
RLRS-F-016	The system shall retry failed email sends and record the final delivery status.	Medium	A temporary SMTP failure is retried; repeated failure is marked Failed and visible to administrators.
RLRS-F-017	The system shall show remote-access instructions for equipment only to the holder of a confirmed reservation for that equipment.	Medium	The holder sees the instructions on the reservation; other users do not.
RLRS-F-018	The system shall allow a lab in-charge to mark a past reservation as Completed or No-show.	Medium	Status is updated and included in reports.
RLRS-F-019	The system shall generate utilisation reports (booked hours versus available hours) per lab and equipment, and booking, cancellation and no-show counts for a selected date range.	Medium	Report figures reconcile with stored reservations for the selected range.
RLRS-F-020	The system shall allow an administrator to configure booking rules, reminder lead time and cancellation cut-off.	Medium	New settings apply to reservations created afterwards.
RLRS-F-021	The system shall record an audit event for reservation changes, approvals, maintenance blocks, user/role changes and settings changes.	High	Audit entries identify action, timestamp, and acting user.
RLRS-F-022	The system shall display	High	User sees a useful message

	validation and operation errors without exposing stack traces or sensitive data.		while technical details remain in protected logs.

5. Non-functional Requirements

Req ID	Requirement	Category	Acceptance / measurement
RLRS-NF-001	For a dataset of 200 equipment items and 20,000 reservations on the project test machine, a one-week calendar view and a booking request should complete within 2 seconds for at least 90% of measured requests.	Performance	Record timings across at least 20 repeated requests; 90% meet target.
RLRS-NF-002	When two users request the same equipment for overlapping times at the same moment, exactly one request shall succeed.	Integrity	Concurrent-request test never produces two overlapping active reservations.
RLRS-NF-003	The application shall preserve committed reservations and settings across restarts, and pending reminder jobs shall resume after a restart without being sent twice.	Reliability	Restart during a test run; records remain and each reminder is sent exactly once.
RLRS-NF-004	When the email service is available, reminder emails shall be handed to it within 5 minutes of their scheduled time.	Timeliness	Compare scheduled and sent timestamps in the email job log.
RLRS-NF-005	Passwords shall not be stored in plaintext; access to protected actions shall require authentication and authorization.	Security	Inspect stored user data and test unauthorized requests.
RLRS-NF-006	The calendar and forms shall provide consistent navigation, readable labels, validation feedback, keyboard operation where supported, and slot states that do not rely on colour alone.	Usability / Accessibility	Complete defined usability checklist for browse, book, cancel and approve flows.
RLRS-NF-007	The codebase shall separate UI, REST routes, the scheduling engine and data persistence so that conflict and booking-rule logic can be unit-tested on its own.	Maintainability	Scheduling-engine tests run without the UI, web server or email service.
RLRS-NF-008	The application shall provide a documented setup procedure and run on the team's supported development environment.	Portability	A team member follows README setup steps on a clean environment and launches the application.

6. Security Objectives and Requirements

6.1 Security Objectives

- Protect user accounts, reservation data and remote-access details from unauthorized viewing or modification.
- Preserve the integrity and traceability of bookings, cancellations, approvals and maintenance blocks so that disputes over a slot can be investigated.

6.2 Security Requirements

Req ID	Requirement	Priority	Acceptance / test

RLRS-SR-001	The system shall require authentication before allowing any reservation to be created, changed or cancelled.	High	Unauthenticated requests to protected endpoints are rejected.
RLRS-SR-002	The system shall enforce role-based authorization in the API/service layer, not only by hiding UI controls.	High	Direct API requests by a role without permission are denied.
RLRS-SR-003	The system shall store passwords using a suitable salted password-hashing mechanism and shall never log passwords.	High	Storage and log inspection show no plaintext passwords.
RLRS-SR-004	The system shall validate and normalize all user-supplied values (IDs, dates, times, text) on the server side.	High	Malformed, missing or out-of-range values are rejected safely.
RLRS-SR-005	The system shall enforce record-level access: students may view and cancel only their own reservations, and lab in-charges may manage only their own labs.	High	Cancelling another user's reservation by direct request is denied.
RLRS-SR-006	SMTP credentials and other secrets shall be supplied through environment configuration and shall not be committed to the repository or written to logs.	High	Repository and log inspection find no secrets.
RLRS-SR-007	The system shall record audit events for reservation and privileged changes without including passwords or other secrets.	High	Audit history records action and actor while excluding secret values.

7. Quality Attributes and Acceptance Tests

Acceptance exit criteria: all high-priority functional requirements have been implemented and tested; no open critical defect remains; no test run produces two overlapping active reservations on the same equipment; and each implemented requirement is linked to at least one test case in the traceability matrix.

Quality attribute	Acceptance test
Correctness	Run conflict boundary cases (overlap, containment, back-to-back) and every booking rule; confirm only valid bookings are stored.
Integrity	Fire simultaneous booking requests for the same slot and confirm exactly one succeeds.
Reliability	Restart with reminders pending and confirm each is sent exactly once.
Security	Test sign-in, role restrictions, record-level access, input validation, and audit-log content.
Performance	Measure week calendar view and booking request against RLRS-NF-001.
Usability	Ask a tester to find free equipment, book it, cancel it, and read the reminder using the UI alone.
Maintainability	Run scheduling-engine unit tests and verify results are repeatable.
Portability	Follow documented setup steps in a clean virtual environment.

8. UML Use-Case Models

Use-case diagram 1 — Booking. Actors: Student, Faculty, Lab In-charge, and Email Service (secondary). Use cases: Sign in; Browse/search equipment; View availability calendar; Create reservation (includes Check conflicts and Check

booking rules); View my reservations; Cancel reservation; Receive confirmation email; Receive reminder email; View remote-access instructions. Lab In-charge additionally approves or rejects restricted bookings and marks attendance.

Use-case diagram 2 — Lab administration and reporting. Actors: Administrator and Lab In-charge. Use cases: Manage users/roles; Manage labs and equipment; Configure operating hours and slot length; Configure booking rules and reminder lead time; Create maintenance block (includes Cancel affected reservations and Notify users); View utilisation report; View no-show report; Review audit history. Students and faculty may view calendars but cannot change equipment, hours or system-wide settings.

9. Requirements Traceability Matrix (RTM)

Req ID	Short requirement	Module	Test case(s)	Status
RLRS-F-001	Sign in/out	Authentication	TC-AUTH-01, TC-AUTH-02	Not tested
RLRS-F-003	Manage equipment	Equipment service/UI	TC-EQ-01, TC-EQ-02	Not tested
RLRS-F-006	Availability calendar	Calendar UI / availability API	TC-CAL-01	Not tested
RLRS-F-007	Create reservation	Reservation service	TC-BOOK-01	Not tested
RLRS-F-008	Conflict detection	Scheduling engine	TC-CONF-01 to TC-CONF-04	Not tested
RLRS-F-009	Booking rules	Scheduling engine	TC-RULE-01 to TC-RULE-04	Not tested
RLRS-F-010	Approval flow	Reservation service	TC-APPR-01	Not tested
RLRS-F-012	Cancel reservation	Reservation service	TC-CANC-01, TC-CANC-02	Not tested
RLRS-F-013	Maintenance block	Scheduling engine	TC-MAINT-01	Not tested
RLRS-F-015	Reminder email	Notification scheduler	TC-MAIL-02, TC-MAIL-03	Not tested
RLRS-F-019	Utilisation report	Reporting module	TC-REP-01	Not tested
RLRS-NF-001	Performance target	Application/API	TC-PERF-01	Not tested
RLRS-NF-002	No double booking under concurrency	Scheduling engine / database	TC-CONC-01	Not tested
RLRS-NF-005	Secure credentials	Authentication	TC-SEC-01	Not tested
RLRS-SR-005	Record-level access	Authorization layer	TC-SEC-02	Not tested